ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А. Э. Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Автоматизация технологического процесса промышленного производства программного обеспечения.

по курсу: Основы программной инженерии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4434		М. Ю. Прохоров
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ 5

ВВЕДЕНИЕ..... 6

1 Концепция и границы информационной системы супермаркета **Error! Bookmark not defined.**

1.1 Описание предметной области ..... **Error! Bookmark not defined.**

1.2 Цель и постановка задачи ..... **Error! Bookmark not defined.**

1.2.1 Границы первой версии..... **Error! Bookmark not defined.**

1.3 Словарь данных предметной области**Error! Bookmark not defined.**

1.4 Нежелательные эффекты и конфликты**Error! Bookmark not defined.**

1.5 Варианты использования ..... **Error! Bookmark not defined.**

1.6 Дерево функций ..... **Error! Bookmark not defined.**

1.7 Заключение и выводы..... **Error! Bookmark not defined.**

2 Спецификация требований к информационной системе супермаркета ..... **Error! Bookmark not defined.**

2.1 Бизнес-требования ..... **Error! Bookmark not defined.**

2.2 Функциональные требования ..... **Error! Bookmark not defined.**

2.2.1 Ограничения и бизнес-правила **Error! Bookmark not defined.**

2.2.2 Внешние интерфейсы ..... **Error! Bookmark not defined.**

2.3 Нефункциональные требования ..... **Error! Bookmark not defined.**

2.3.1 Информационные требования к UI**Error! Bookmark not defined.**

2.3.2 Функциональные требования к UI**Error! Bookmark not defined.**

2.4 Заключение и выводы..... **Error! Bookmark not defined.**

3 Модель предметной области..... **Error! Bookmark not defined.**

3.1 Диаграмма вариантов использования**Error! Bookmark not defined.**

3.2 Диаграмма объектов ..... **Error! Bookmark not defined.**

3.3 Бизнес-процесс продажи на кассе... **Error! Bookmark not defined.**

3.4 Диаграмма состояний ..... **Error! Bookmark not defined.**

3.5 ER-модель ..... **Error! Bookmark not defined.**

3.6 Заключение и выводы..... **Error! Bookmark not defined.**

3.7 Проверка машинной модели..... **Error! Bookmark not defined.**

4 Архитектура программного обеспечения**Error! Bookmark not defined.**

4.1 Вариант архитектуры и обоснование**Error! Bookmark not defined.**

4.1.1 Модульная структура ..... **Error! Bookmark not defined.**

4.1.2 Компоненты и соединители..... **Error! Bookmark not defined.**

4.2 Структуры распределения..... **Error! Bookmark not defined.**

4.2.1 Размещение..... **Error! Bookmark not defined.**

4.2.2 Реализация ..... **Error! Bookmark not defined.**

4.2.3 Распределение функций между участниками..... **Error! Bookmark not defined.**

4.3 Надёжность и эксплуатационные границы**Error! Bookmark not defined.**

4.4 Заключение и выводы..... **Error! Bookmark not defined.**

5 Роли и защита ЛР №1 ..... **Error! Bookmark not defined.**

ЗАКЛЮЧЕНИЕ 13

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 14

ЗАДАНИЕ

Разработка спецификации требований в соответствии с вариантом задания. Формализация ограничений и бизнес-правил предметной области.

Моделирование предметной области и архитектуры программного обеспечения с использованием нотаций UML, BPMN, ER-диаграмм и других языков моделирования по необходимости и согласованию с преподавателем.

Выполнение индивидуального раздела лабораторной работы в роли: «Архитектор программного обеспечения».

взаимодействие в группе из трех человек в ролях: «Специалист по научноисследовательским и опытно-конструкторским разработкам», «Системный

ВВЕДЕНИЕ

Работа посвящена требованиям и проектированию информационной системы супермаркета по варианту 36. Концепция, ограничения, модели и архитектура задают проверяемый контракт будущей реализации. Исполняемый прикладной код на этом этапе не разрабатывается.

Основание выполнения — соответствующая методичка [1]. Порядок оформления сверяется с нормативной документацией ГУАП [2].

1 Архитектура программного обеспечения

1.1 Вариант архитектуры и обоснование

Выбрана трёхуровневая клиент-серверная архитектура: React-клиент в браузере, FastAPI-сервер приложений, PostgreSQL. Nginx отдаёт статические файлы и проксирует /api. Предметные функции объединены в модульный монолит: для одной команды из трёх человек это позволяет сохранять продажу одной транзакцией и не вводить распределённые транзакции. Стек соответствует согласованному отступлению от примеров методички; официальная документация React, FastAPI и PostgreSQL использована как технический справочник [9–11].

1.1.1 Модульная структура

Таблица 8 — Модули, методы и данные проектируемой системы

Модуль	**Назначение и методы в псевдокоде**	**Данные и зависимости**
Auth	login(credentials), me(token), logout(token), requireRole(role)	Сотрудник, сеанс; использует Security и DB
Security	hashPassword(password), verifyPassword(password, hash), hashToken(token)	Соль и хеш; не зависит от UI
Catalog	list(query,cursor,limit), create(card), update(card,version)	Товар; использует Auth, Validation, DB
Inventory	adjust(product,delta,reason), recordMovement(...)	Остаток и движение; использует Auth, DB
Sales	createReceipt(key,lines), pay(id), cancel(id), read(id), history(cursor)	Чек, строка; использует Auth, Catalog/Inventory через общую транзакцию
Validation	validateProduct(...), validateReceipt(...)	Схемы запросов; ограничивает типы, диапазоны и повтор товаров
DB	acquireConnection(), begin(), commit(), rollback()	Пул соединений и параметризованные SQL-запросы
Web Catalog	search(), nextPage(), addToCart(), editCard()	Состояние фильтра, текущая страница; вызывает API-client
Web Checkout	prepare(), confirmCash(), cancelDraft()	Корзина, серверный чек, ключ повтора
API-client	request(path,body,token), validateResponse(...)	JSON, отмена запроса, преобразование ошибок

Декомпозиция по функциям; наследование не служит основным способом организации приложения. Схемы запросов могут наследовать общие правила Pydantic. React-компоненты составляются композиционно и используют хуки.

1.1.2 Компоненты и соединители

Браузер инициирует HTTP-запросы; Nginx отдаёт сборку и передаёт API-запросы FastAPI. Сервер выполняет аутентификацию, валидацию и бизнес-правила, затем обращается к БД по PostgreSQL wire protocol. Совместно используемые данные находятся только в PostgreSQL. UI не открывает соединение с БД.

Параллелизм: несколько браузеров и несколько обработчиков API используют общий пул. При оплате сервер блокирует чек, затем товары в возрастающем порядке ID. Такой порядок уменьшает возможность взаимоблокировки. Строки разных товаров допускают параллельные продажи; конфликт за последний экземпляр разрешается БД. READ COMMITTED с блокировками достаточен для выбранной транзакции; повторная проверка после получения блокировки обязательна.

Псевдокод оплаты: проверить сеанс → начать транзакцию → заблокировать чек → вернуть результат, если уже paid → проверить draft → заблокировать товары по ID → проверить остаток и активность → уменьшить остаток → создать движения → пометить paid → commit. Ошибка на любом шаге приводит к rollback. Сетевой ответ отправляется после фиксации транзакции.

1.2 Структуры распределения

1.2.1 Размещение

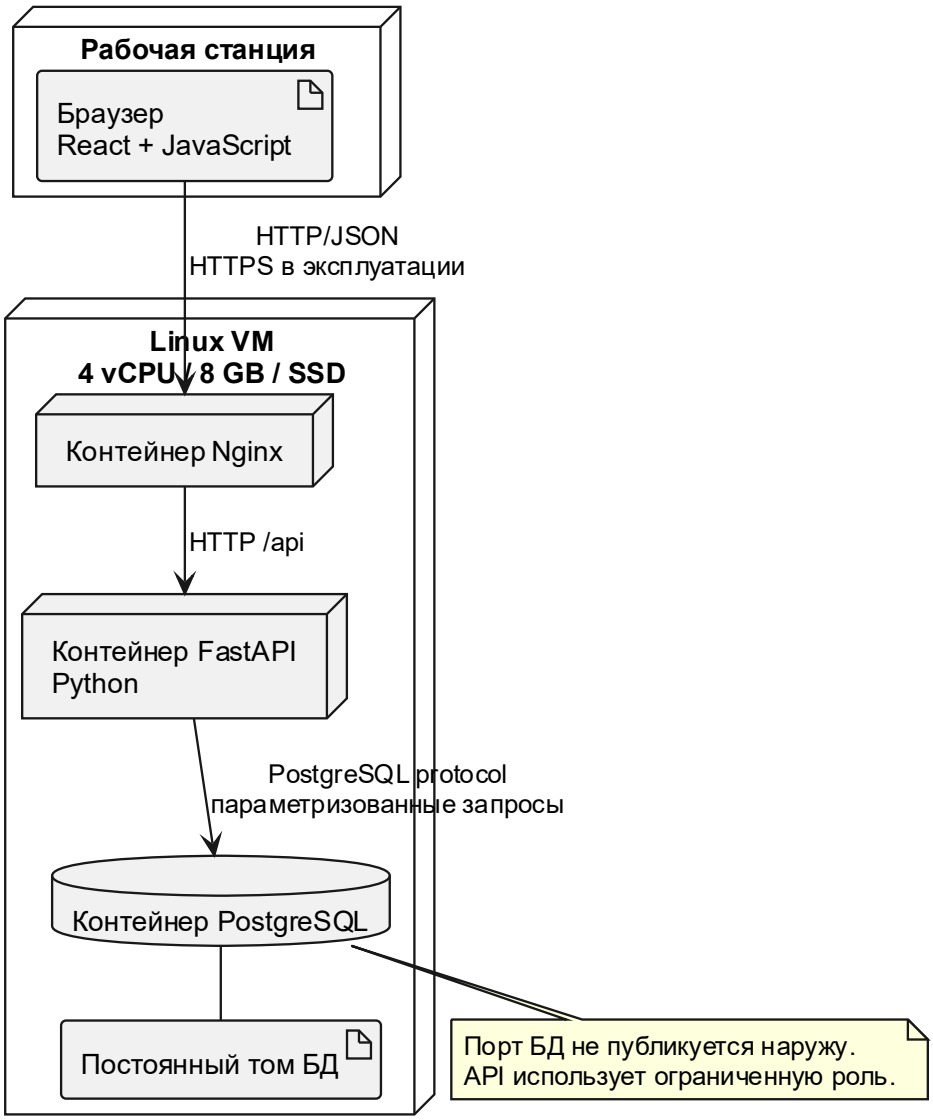


Рисунок 1 — схема физ. размещения компонентов системы

Рабочая станция кассира выполняет браузер. Linux VM выполняет Nginx, FastAPI и PostgreSQL в отдельных контейнерах. Том БД сохраняется между перезапусками. База и API не публикуют порты на внешний интерфейс; доступ к приложению в учебном стенде ограничен localhost. Для доступа из локальной сети адрес публикации и TLS настраиваются отдельно.

Проектная VM: 4 vCPU, 8 ГБ RAM, 40 ГБ диска с возможностью расширения; гостевая ОС Linux, сеть NAT с ограниченным пробросом или host-only. Производительность проверяется измерением на реальном

выделенном стенде. Контейнеры не заменяют этап установки VM, предусмотренный ЛР №2.

1.2.2 Реализация

Таблица 9 — План отображения модулей на файлы будущей реализации

Модуль	**Планируемое место реализации на этапе ЛР №3**
Сборка приложения и lifecycle	backend/app/main.py
HTTP-маршруты Auth, Catalog, Sales	backend/app/routers/auth.py, products.py, receipts.py
Предметные операции Catalog, Inventory, Sales	backend/app/services/products.py, receipts.py
Аутентификация и проверка ролей	backend/app/dependencies.py
Преобразование ошибок в HTTP-ответы	backend/app/errors.py
Validation	backend/app/schemas.py
Security	backend/app/security.py
DB	backend/app/db.py
Схема и права БД	init.sql, db/pg_hba.conf
Web Catalog	frontend/src/Catalog.jsx
Оркестрация состояния интерфейса	frontend/src/App.jsx
Вход, навигация, редактор товара	frontend/src/components/*.jsx
Checkout и история	frontend/src/features/checkout/*.jsx, frontend/src/features/receipts/*.jsx
API-client	frontend/src/api.js
Стили	frontend/src/styles.css
Развёртывание прикладной версии	docker-compose.yml, Dockerfile компонентов

Это план структуры будущего кода. В ЛР №1 размещаются только документы и модели. ЛР №2 создаёт отдельный инфраструктурный стенд с заглушкой и своим томом; прикладная схема и исходники появляются только в ЛР №3. ЛР №4 реализует отдельную инструментальную утилиту варианта 12.

1.2.3 Распределение функций между участниками

Таблица 10 — Обязанности по архитектуре и интеграции

Участник	Роль в ЛР №1	Ответственность	Результат и форма отчетности
A	Специалист по НИОКР	Концепция, требования, ограничения и критерии	Разделы 1 и 2 отчёта (концепция, словарь данных, спецификация требований BR/FR/NFR)
B	Системный аналитик	UC, объекты, BPMN, состояния, ER; согласование терминов	Раздел 3 отчёта (диаграммы Use Case, объектов, состояний, ER-модель, процесс BPMN)
C	Архитектор ПО	Модули, протоколы, транзакции, размещение и план реализации	Раздел 4 отчёта (модули, компоненты и соединители, структуры размещения и план реализации)

На этапе реализации A принимает GUI, B реализует API и интеграцию, C отвечает за БД. A — Кочетков Роман; B — Энгельгард Кирилл; C — Прохоров Максим. Обмен контрактом JSON и согласование бизнес-правил предшествуют реализации; изменения требований оформляются задачей и проходят совместный обзор.

1.3 Надёжность и эксплуатационные границы

Блокировки обеспечивают согласованность между экземплярами API, а не только внутри одного процесса. Размер пула ограничивает число соединений. Запросы имеют таймаут; занятый сервис отвечает временной ошибкой, которую клиент может повторить. Отсутствие сети не разрешает клиенту списывать остатки самостоятельно.

Секреты передаются через локальный конфигурационный файл, не включаются в сборку клиента и Git. Различаются пользовательские роли приложения и технические роли БД. Администратор БД доверенный и может обходить прикладные ограничения; его действия выполняются по регламенту. Контур приложения и контур Redmine имеют разные базы и тома.

1.4 Заключение и выводы

1. Три уровня разделяют отображение, принятие решений и сохранение данных.

2. Модульный монолит соответствует размеру команды и общей транзакции продажи.

3. Структуры размещения, реализации и распределения обязанностей описывают разные стороны одной архитектуры.

1. Масштабирование начинается с корректных запросов, индексов и ограниченных ответов; отдельные сервисы вводятся только при подтверждённой необходимости.

ЗАКЛЮЧЕНИЕ

Сформированы концепция, словарь, конфликты, дерево функций, спецификация и CRUD-матрица. Подготовлены UML-модели, BPMN и ER-модель. Определены клиент-серверные компоненты, правила транзакций и структуры размещения, реализации и распределения обязанностей. Модели проверены программной отрисовкой; первая работа сохранена как самостоятельный этап проектирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии. Методические указания к выполнению лабораторной работы № 1 / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург : ГУАП, 2021. Электронный документ, предоставленный к заданию.

2. ГУАП. Для учебного процесса. Нормативная документация. URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 03.10.2026).

3. ISO/IEC/IEEE 29148:2018. Systems and software engineering — Requirements engineering. URL: https://www.iso.org/standard/72089.html (дата обращения: 03.10.2026).

1. OMG. Unified Modeling Language (UML), Version 2.5.1. URL: https://www.omg.org/spec/UML/2.5.1/ (дата обращения: 03.10.2026).

5. OMG. Business Process Model and Notation (BPMN), Version 2.0.2. URL: https://www.omg.org/spec/BPMN/2.0.2/PDF (дата обращения: 03.10.2026).

6. PlantUML. Language Reference Guide. URL: https://plantuml.com/guide (дата обращения: 03.10.2026).

7. bpmn-io. bpmn-js: BPMN 2.0 rendering toolkit and web modeler. URL: https://github.com/bpmn-io/bpmn-js (дата обращения: 03.10.2026).

8. n8n. Documentation: workflow automation and integrations. URL: https://docs.n8n.io/ (дата обращения: 03.10.2026).

9. React. Learn React. URL: https://react.dev/learn (дата обращения: 03.10.2026).

10. FastAPI. Tutorial. URL: https://fastapi.tiangolo.com/tutorial/ (дата обращения: 03.10.2026).

11. PostgreSQL Global Development Group. PostgreSQL 17 Documentation. URL: https://www.postgresql.org/docs/17/ (дата обращения: 03.10.2026).